

Name: Arfa Faheem

RegNo: 23-NTU-CS-1247

Embedded IoT Systems (CSE-3080)

Fall 2025

(BSAI-5)

Assignment 1

Question 1 — Short Questions

Answer concisely, in your own words.

1. Why is volatile used for variables shared with ISRs?

Volatile is used for variables shared with ISRs because it makes the program to always reads the latest value.

As the compiler without volatile keyword will take the previous value of variable and don't update with new one most of the time.

2. Compare hardware-timer ISR debouncing vs. delay()-based debouncing.

In delay() based debouncing, we use delay(10) function to stop the program for a short time so that button can settle and code gets its stable state (ON/OFF) but it stops the other programs and don't let the code run.

Hardware timer ISR debouncing, a timer interrupt checks the button's state after some time period(in ms) and let the code know that the button is stable but it doesn't stop the code remaining execution so its faster.

3. What does IRAM_ATTR do, and why is it needed?

IRAM_ATTR is Internal Ram which puts the function ISR in ram not flash. It is used before ISR function (handlebutton, timer etc) which handles the interrupt and it is needed for faster and efficient handling when the flash memory is busy.

4. Define LEDC channels, timers, and duty cycle.

LEDC channels get the PWM signal from the pin to which it is connected so basically they are used for getting PWM signal. Esp has 16 channels.

LEDC timers control frequency and resolution of PWM signal.

Duty Cycle handles the duration of ON/OFF state that how long the PWM cycle stays bright or dim. For example: 70% Bright(on) and 30% Dim(off) etc

5. **Why should you avoid Serial prints or long code paths inside ISRs?**

ISR needed to run very quickly as it handles interrupts so when serial prints or long code paths are used then it will take more execution time and ISR can miss the interrupts so these should be avoided for smoother and faster flow of ISR working.

6. **What are the advantages of timer-based task scheduling?**

It assigns a specific time interval to every task so that the tasks can run efficiently without disturbing the execution of other tasks.

This makes the flow faster and avoid task waiting.

7. **Describe I²C signals SDA and SCL.**

Inter-Integrated circuit is a communication protocol which uses two wires SDA,SCL

SDA (Serial Data Line) is for carrying data between devices on pin 21, It works on master-slave behavior.

SCL is for clock on pin 22. It provides clock signals.

8. **What is the difference between polling and interrupt-driven input?**

Polling is a method in which microcontroller checks the input repeatedly in a loop. This wastes CPU time and important events can miss. But in interrupt driven input, the microcontroller performs its task on normal basis. It doesn't check the input but responds

only when an interrupt occurs. In short, polling constantly checks for interrupts but interrupt driven responds only when event has occurred.

9. What is contact bounce, and why must it be handled?

When a hardware switch (button) is pressed but not cleanly and it bounces for a few ms, like high->low->high->low confusing the digitalRead function about the state. The microcontroller may take it as pressing button multiple times. It must be handled by debouncing for better flow and stability so correct button state can be read.

10. How does the LEDC peripheral improve PWM precision?

LEDC is a built in hardware peripheral which uses hardware timers to generate PWM signals for controlling leds, sensors etc. It uses timer to provide accurate and smoother signals and faster as it is independent of the CPU. In this way, it improves PWM precision.

11. How many hardware timers are available on the ESP32?

There are total 8 hardware timers (in 2 groups) in esp32. Each group has 4 timers (Timer_0,1,2,3)

12. What is a timer prescaler, and why is it used?

Timer prescaler is divider that reduces the frequency of clock signal by dividing it. The esp's clock is very fast that it can overflow and we can't even notice the time intervals. Prescaler slows down it like 80MHz to 1MHz allowing accurate timing.

13. Define duty cycle and frequency in PWM.

Duty cycle handles the ON/OFF state and determines the duration of the PWM cycle in percentage that how long will it off and how long on. For example, 100% duty cycle fully bright and 0% full dim like off, 70% means 70% bright and 30%dim.

Frequency refers to number of PWM cycles completed per second. 5MHz means 5×10^6 cycles per second is completed.

14. How do you compute duty for a given brightness level?

To compute duty cycle for a given brightness level, following formula is used:

- Technical formula is:

Duty Cycle (%) = $\frac{TON + TOFF}{TON} \times 100$

- Programmatically for esp32 it is:

Duty Value = $\frac{\text{Brightness (\%)}}{100} \times \text{Max Duty Value}$

15. Contrast non-blocking vs. blocking timing.

In blocking timing, program stops completely for a short period of time like in delay(10) and other execution occurs. This consumes time and slows down the program.

Non blocking timing doesn't stop the whole program, it allows the program to run and track time in the background. It is faster and efficient eg: millis()

Hardware based non blocking timing also includes ISR which only runs when time is reached.

16. What resolution (bits) does LEDC support?

Common are 8 bits or 10 bits. But it supports 1-16 bits.

17. Compare general-purpose hardware timers and LEDC (PWM) timers.

General purpose timers are used for scheduling tasks, triggering interrupts at specific intervals etc and esp32 has 8 hardware timers (4x2).

LEDC PWM timers are used for generating PWM signals. It control frequency and resolution of PWM signals. Esp32 has 4 LEDC timers.

18. What is the difference between Adafruit_SSD1306 and Adafruit_GFX?

Adafruit_GFX is used for graphics. This library provides various graphical functionalities like draw line, draw shapes, writing text etc. While Adafruit-SSD1306 is for OLED driver that handles communication, used with Adafruit_gfx.

19. How can you optimize text rendering performance on an OLED?

We can optimize text rendering performance on OLED by using small fonts, avoiding unnecessary clear functions and redrawing everything, use `display.display()` functions when data changed.

20. Give short specifications of your selected ESP32 board (NodeMCU-32S).

- Processor is Xtensa dual-core 32-bit LX6 microprocessor.
- Clock frequency is 80 to 240 MHz
- 2.4 GHz WiFi and Bluetooth
- 520 KB SRAM
- 30 GPIO pins
- 4 hardware timers (2 groups)
- Support timer, interrupt, debouncing, I2C, PWM etc

Question 2 — Logical Questions

1. A 10 kHz signal has an ON time of 10 ms. What is the duty cycle? Justify with the formula.

The formula for the duty cycle is:

$$\text{Duty Cycle (\%)} = \text{TON} / \text{TOFF} \times 100$$

Frequency is 10kHz so time period is 0.1ms. ON time is 10ms which is larger than period it should be less than or equal to PWM period. So practically its not possible

It will be $10 / 0.1 \times 100 = 10,000\%$ duty cycle

2. How many hardware interrupts and timers can be used concurrently? Justify.

Esp32 has 4 hardware timers (2 groups) other 4 are ledc timers. So these timers can generate their own interrupts independently. The interrupt controller can handles them in such a way that multiple timers and hardware run concurrently.

3. How many PWM-driven devices can run at distinct frequencies at the same time on ESP32? Explain constraints.

PWM has 16 channels in esp32 so it can control 16 devices at a time with shared frequency. But there are 4 LEDC timers so for distinct frequency, only 4 devices can run at a time others have to share frequency. Frequency is limited by number of timers.

4. Compare a 30% duty cycle at 8-bit resolution and 1 kHz to a 30% duty cycle at 10-bit resolution (all else equal).

At 8-bit resolution: $30\% \times 256$ as bits are from 0-255 which is 77 approx.

At 10-bit resolution: $30\% \times 1024$ as bits are 0-1023 which actual duty is 307 approx.

10-bit resolution provides more smoother and fine PWM signals brightness levels.

5. How many characters can be displayed on a 128x64 OLED at once with the minimum font size vs. the maximum font size? State assumptions.

Minimum font size is 5x7.

Maximum font size is 16x32.

Total characters = (OLED width / font width) $\times$ (OLED height / font height)

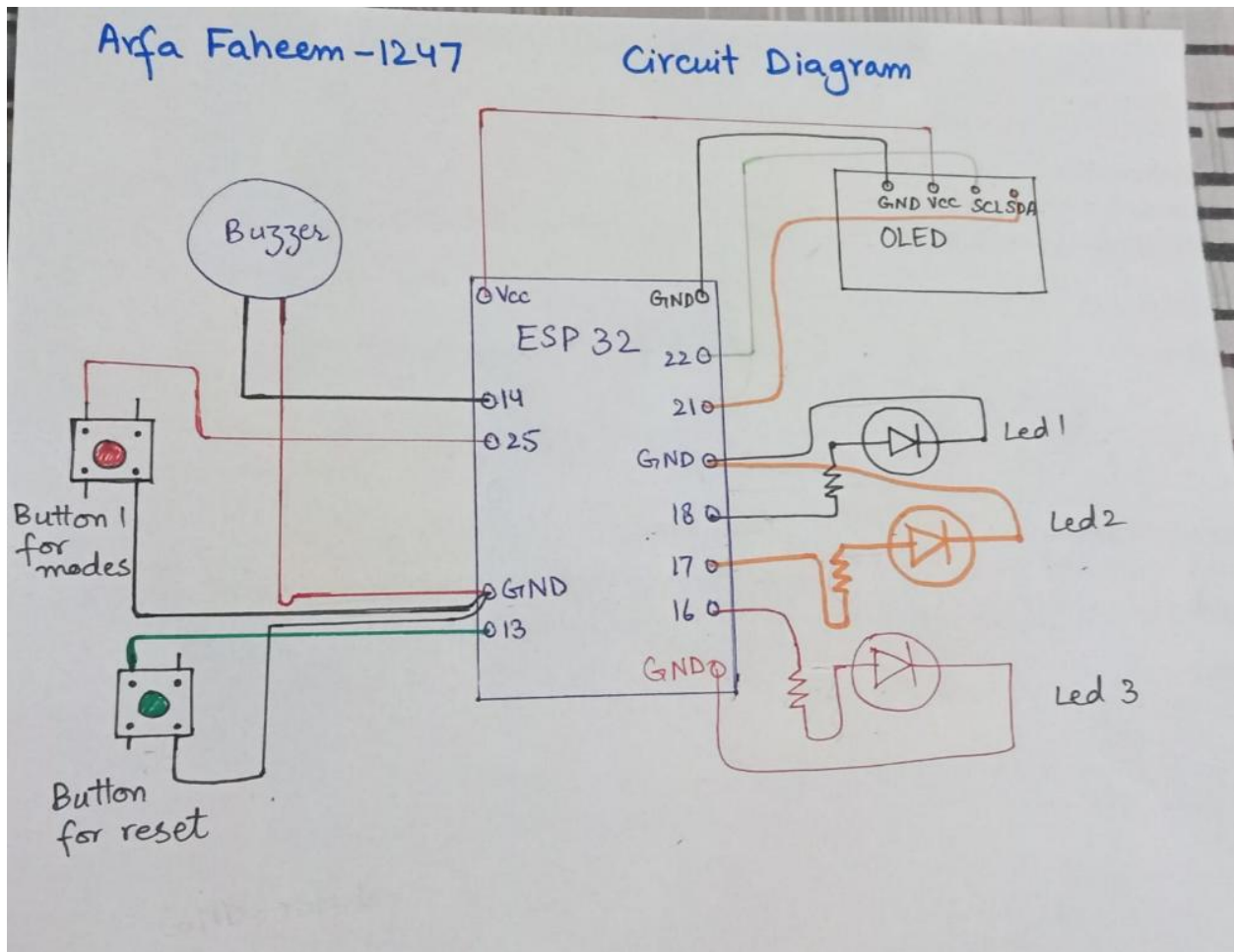
Total character for minimum font is 255 and for maximum is 16.

Below is whole calculation:

The image shows handwritten calculations on lined paper. At the top, 'Max font (16 x 32)' is written with 'width' above the 16 and 'height' above the 32. An arrow points from this to 'OLED (128 x 64)'. Below this, the calculation for total characters is shown:
$$\text{Total characters} = \left(\frac{128}{16} \right) \times \left(\frac{64}{32} \right) = 8 \times 2 = 16$$
 Then, 'Min font (5 x 7)' is written. Below that, the calculation for total characters is shown:
$$\text{Total characters} = \left(\frac{128}{5} \times \frac{64}{7} \right) = 25 \times 9 = 225$$
 A small 'axy A04' is visible in the bottom left corner of the paper.

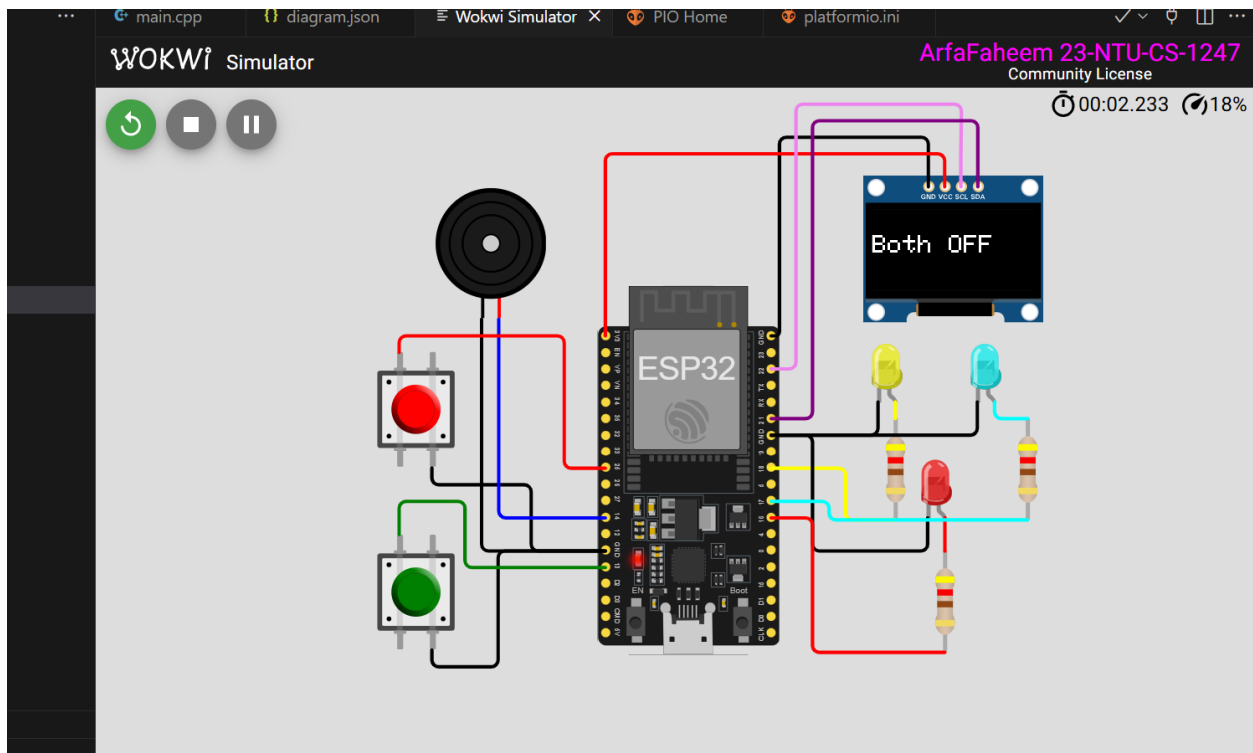
Question 3 — Implementation

Circuit Diagram:

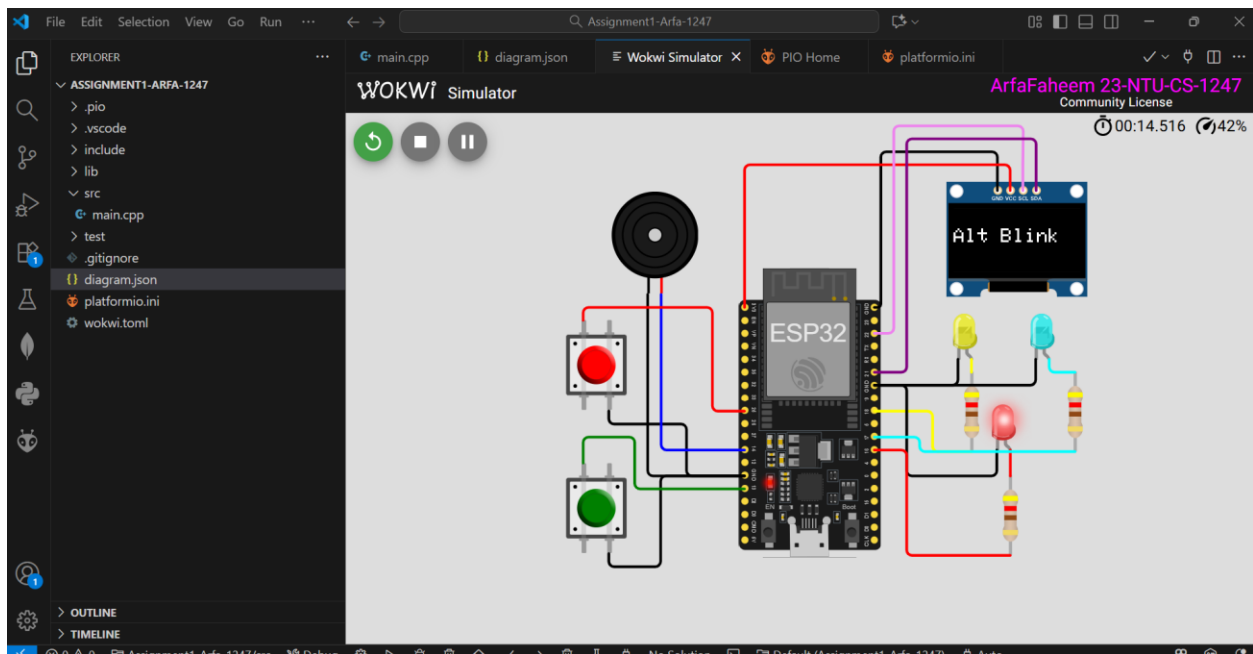


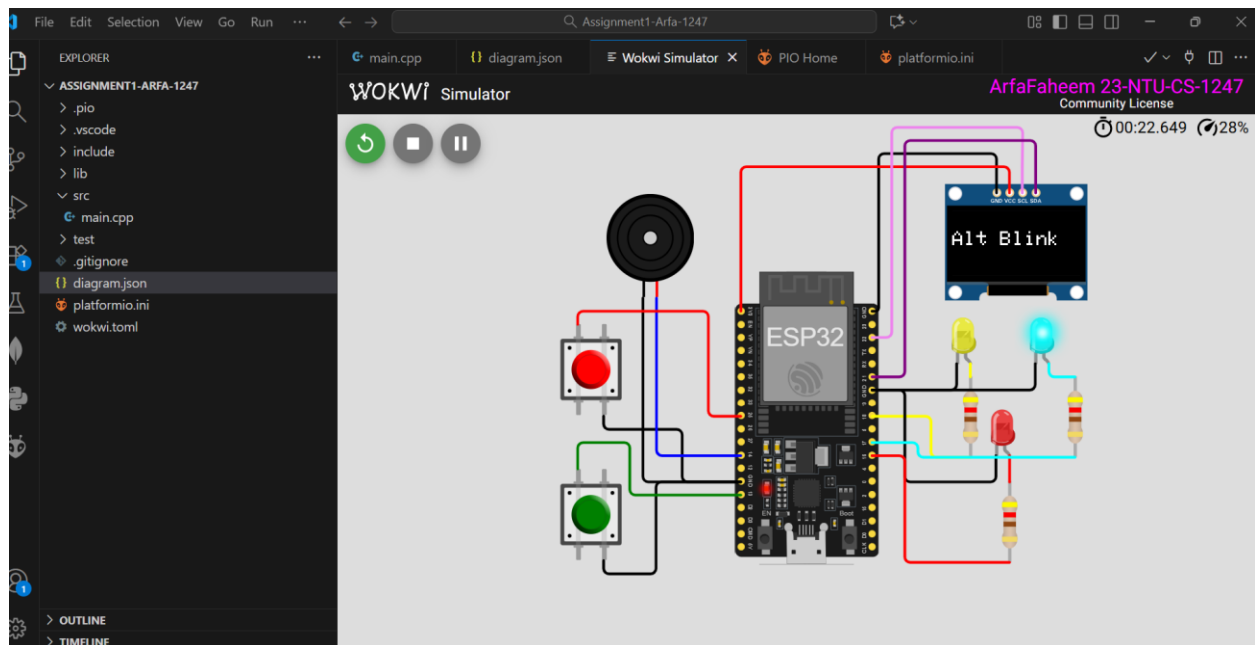
Task A — Coding: Use one button to cycle through LED modes (display the current state on the OLED)

Mode 1- Both OFF

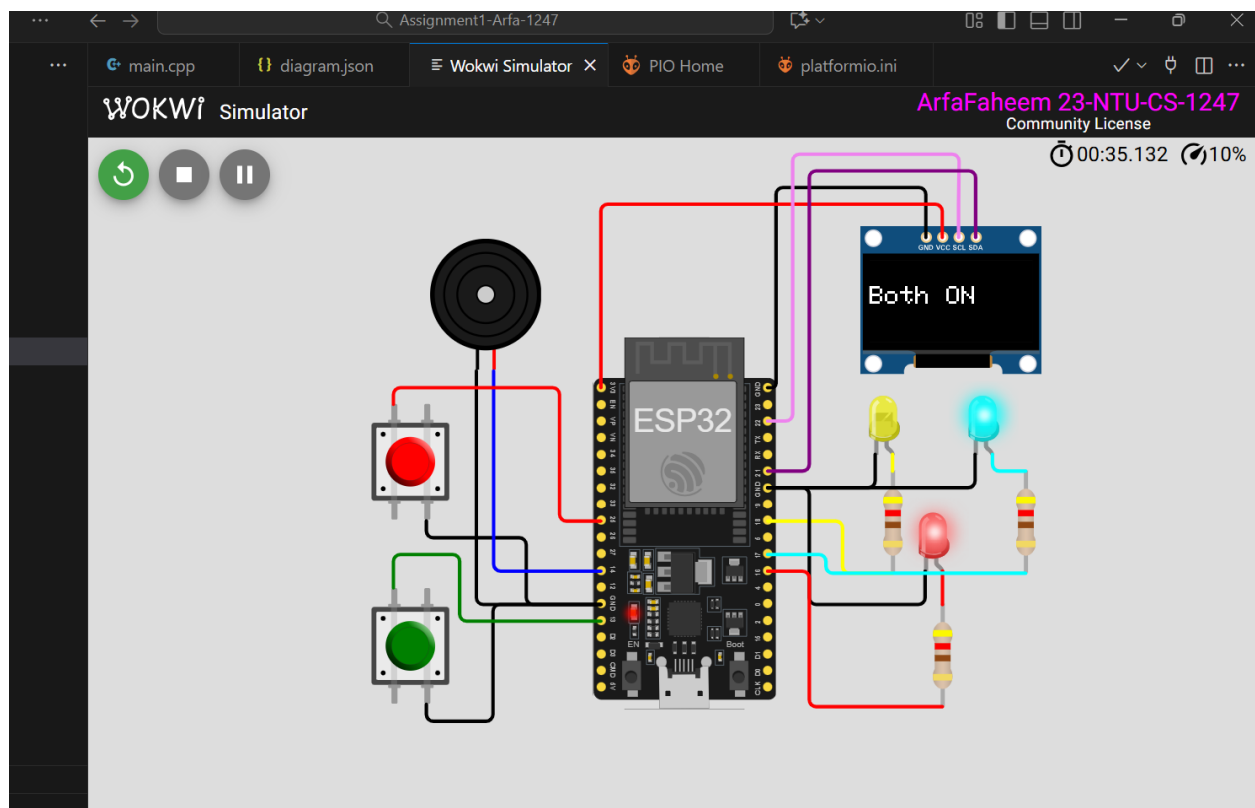


Mode 2- Alternate Blink

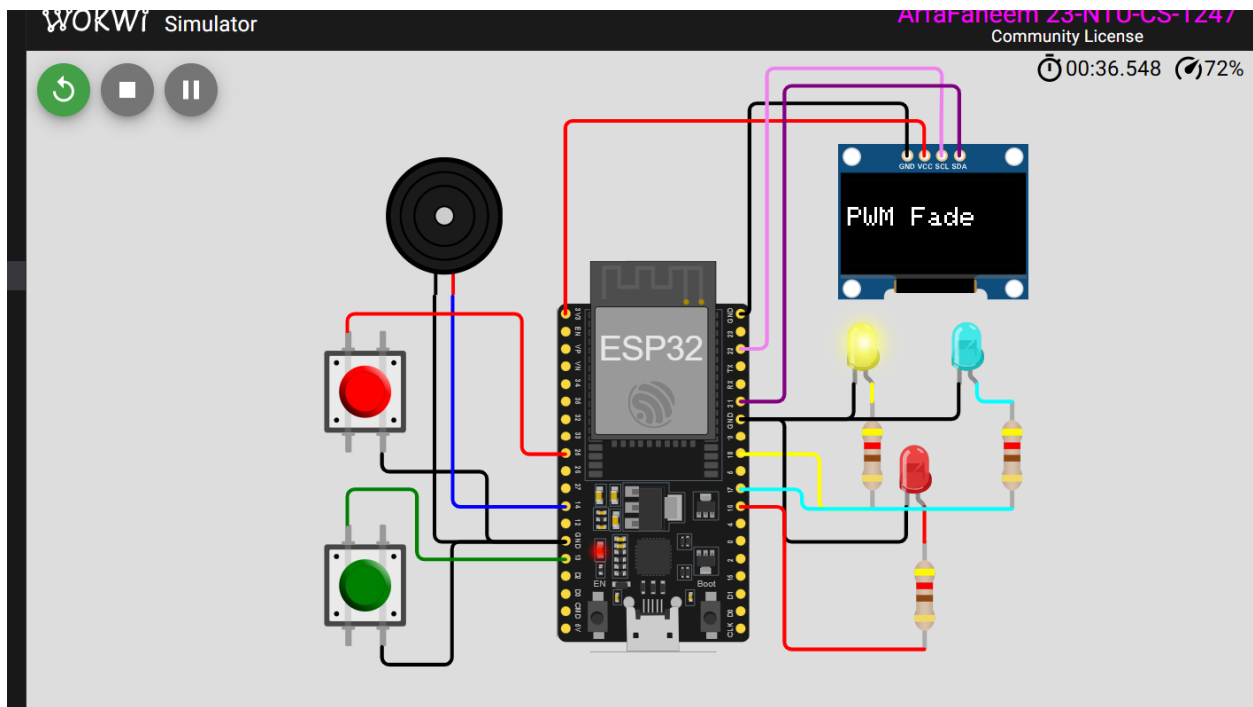




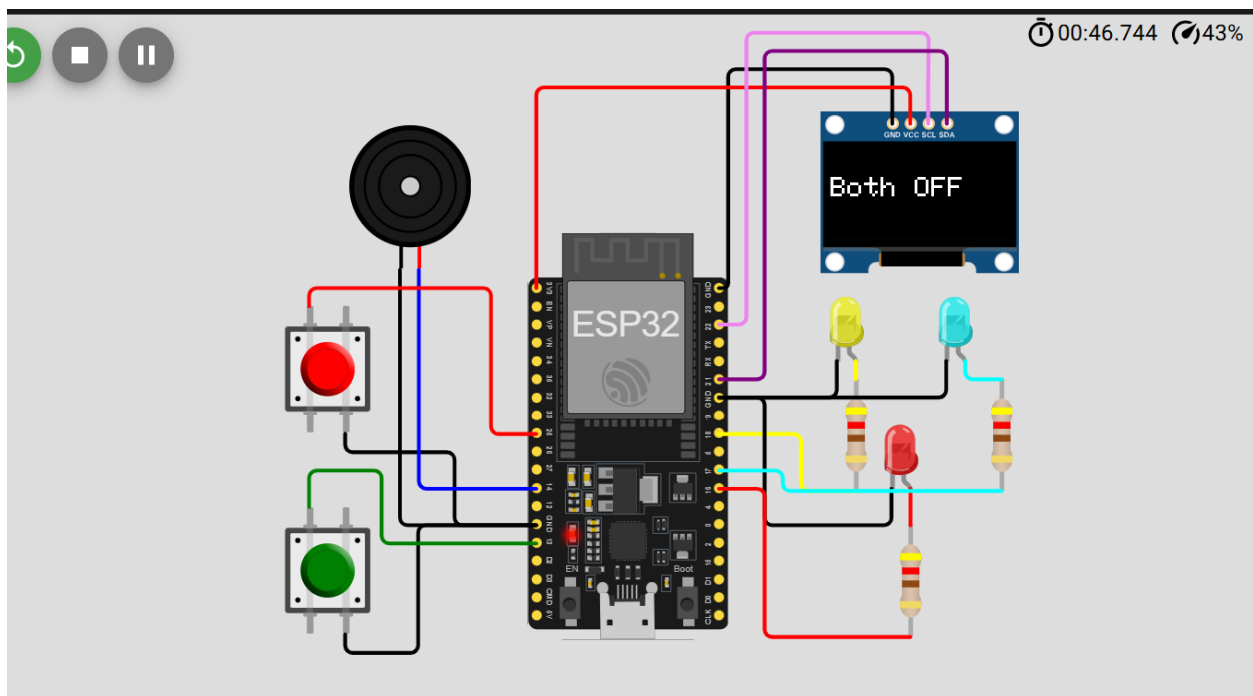
Mode 3- Both ON



Mode 4- PWM Fade



Reset



Code for task A:

```
//Task A
```

```
//Name: Arfa Faheem RegNo: 23-NTU-CS-1247
```

```
//Embedded IOT Fall 2025
```

```
//BS AI 5TH
```

```
//Submitted to: Sir Nasir
```

```
#include <Arduino.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
//Here i configure pins the red button is on 25 which is controlling all modes
```

```
#define BTN_MODE 25
```

```
#define BTN_RESET 13
```

```
#define LED1 16
```

```
#define LED2 17
```

```
#define LED3 18
```

```
//this is for oled configuration
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_ADDR 0x3C
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

```
//this is for PWM, channel 0 for led3 and frequency is 5000,
```

```
//resolution is set to 10 bit for smoothness
```

```
#define LED3_CH 0
```

```
#define PWM_FREQ 5000
```

```
#define PWM_RES 10
```

```
int mode = 0; // Mode 0: OFF, Mode 1: Alternate Blink, ,Mode 2: Both ON, Mode 3: PWM  
Fade
```

```
unsigned long lastModePress = 0;
```

```
unsigned long lastResetPress = 0;
```

```
const unsigned long debounceDelay = 250; // debounce time is in ms
```

```
void showModeOnOLED() { //this function is to show mode on oled display
```

```
    display.clearDisplay();
```

```
    display.setTextSize(2);
```

```
    display.setTextColor(SSD1306_WHITE);
```

```
    display.setCursor(0, 20);
```

```
    if (mode == 0) display.println("Both OFF");
```

```
    else if (mode == 1) display.println("Alt Blink"); //alternate blink means led 1 on, led 2 off and  
vice versa
```

```
    else if (mode == 2) display.println("Both ON");
```

```
    else if (mode == 3) display.println("PWM Fade"); //pwm fade means led3 will fade high and  
low
```

```
    display.display();
```

```
}
```

```
void setup() {
```

```
    Serial.begin(115200);
```

```
    // here pin mode is set for buttons, leds
```

```
pinMode(BTN_MODE, INPUT_PULLUP);
pinMode(BTN_RESET, INPUT_PULLUP);
pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);

// PWM setup for LED3
ledcSetup(LED3_CH, PWM_FREQ, PWM_RES);
ledcAttachPin(LED3, LED3_CH);

// Initialize OLED
Wire.begin(21, 22);
if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("OLED initialization failed");
    while (1);
}

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0, 0);
display.println("Task A - LED Modes"); //This is start of oled
display.display();
delay(1000);
//this will call the function to display current mode on oled
showModeOnOLED();
}
```

```

void loop() {

    unsigned long currentTime = millis(); //used for debouncing, it takes current time in
    milliseconds

    //this is for debouncing to prevent multiple button press changes

    if (digitalRead(BTN_MODE) == LOW && (currentTime - lastModePress > debounceDelay)) {
        //current time in milliseconds by millis() function

        lastModePress = currentTime;

        mode = (mode + 1) % 4; // it will handle Cycle 0 to 3

        showModeOnOLED();
    }

    //here, i handle debounce with reset button to go to mode 0 where both leds off

    if (digitalRead(BTN_RESET) == LOW && (currentTime - lastResetPress > debounceDelay)) {

        lastResetPress = currentTime;

        mode = 0;

        showModeOnOLED();
    }

    if (mode == 0) { // Both OFF

        digitalWrite(LED1, LOW);

        digitalWrite(LED2, LOW);

    }
}

```

```
else if (mode == 1) { // Alternate Blink
```

```
    digitalWrite(LED1, HIGH);
```

```
    digitalWrite(LED2, LOW);
```

```
    delay(700);
```

```
    digitalWrite(LED1, LOW);
```

```
    digitalWrite(LED2, HIGH);
```

```
    delay(700);
```

```
}
```

```
else if (mode == 2) { // Both ON
```

```
    digitalWrite(LED1, HIGH);
```

```
    digitalWrite(LED2, HIGH);
```

```
    ledcWrite(LED3_CH, 0);
```

```
}
```

```
else if (mode == 3) { // PWM Fade on LED3 only
```

```
    digitalWrite(LED1, LOW); //led 1 and 2 will be off in this mode
```

```
    digitalWrite(LED2, LOW);
```

```
    for (int duty = 0; duty <= 1023; duty += 5) { //duty cycle is with 5 small steps for more  
smoothing
```

```
        ledcWrite(LED3_CH, duty);
```

```
        delay(7);
```

```
    }
```

```
    for (int duty = 1023; duty >= 0; duty -= 5) {
```

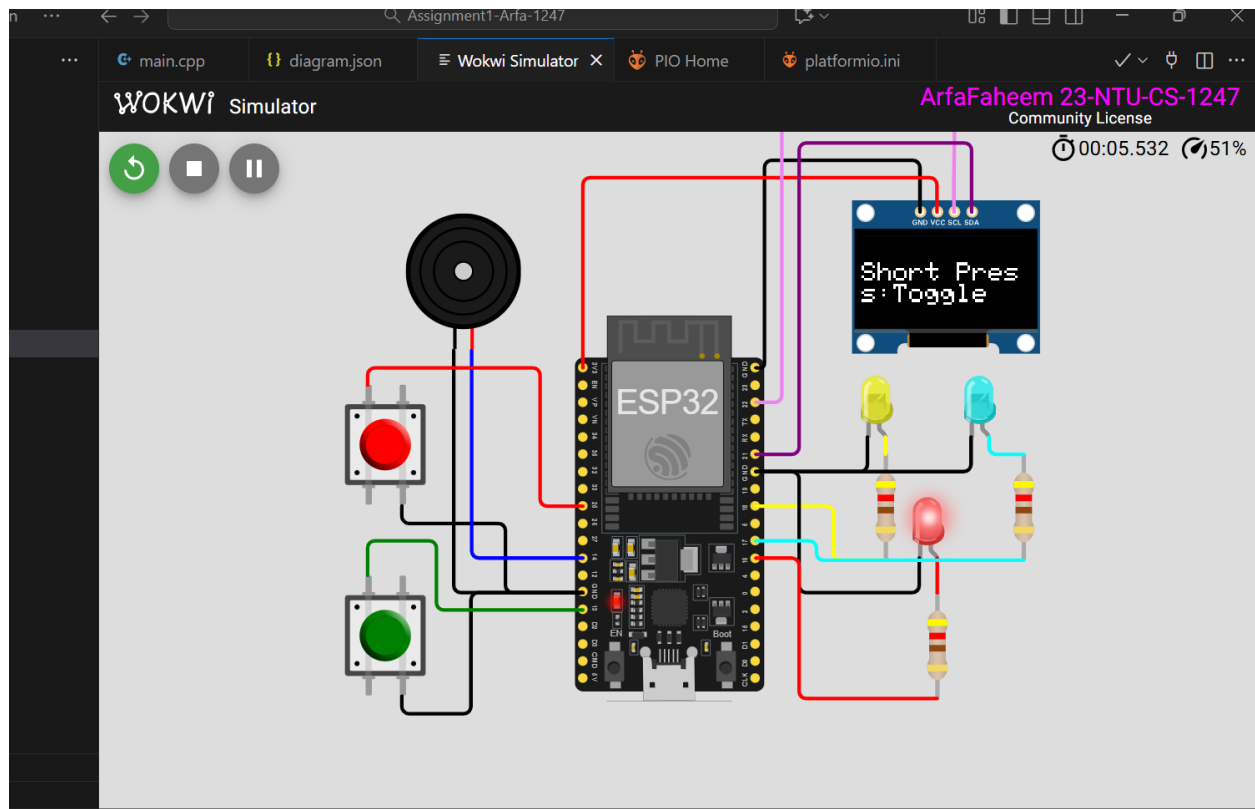
```
        ledcWrite(LED3_CH, duty);
```

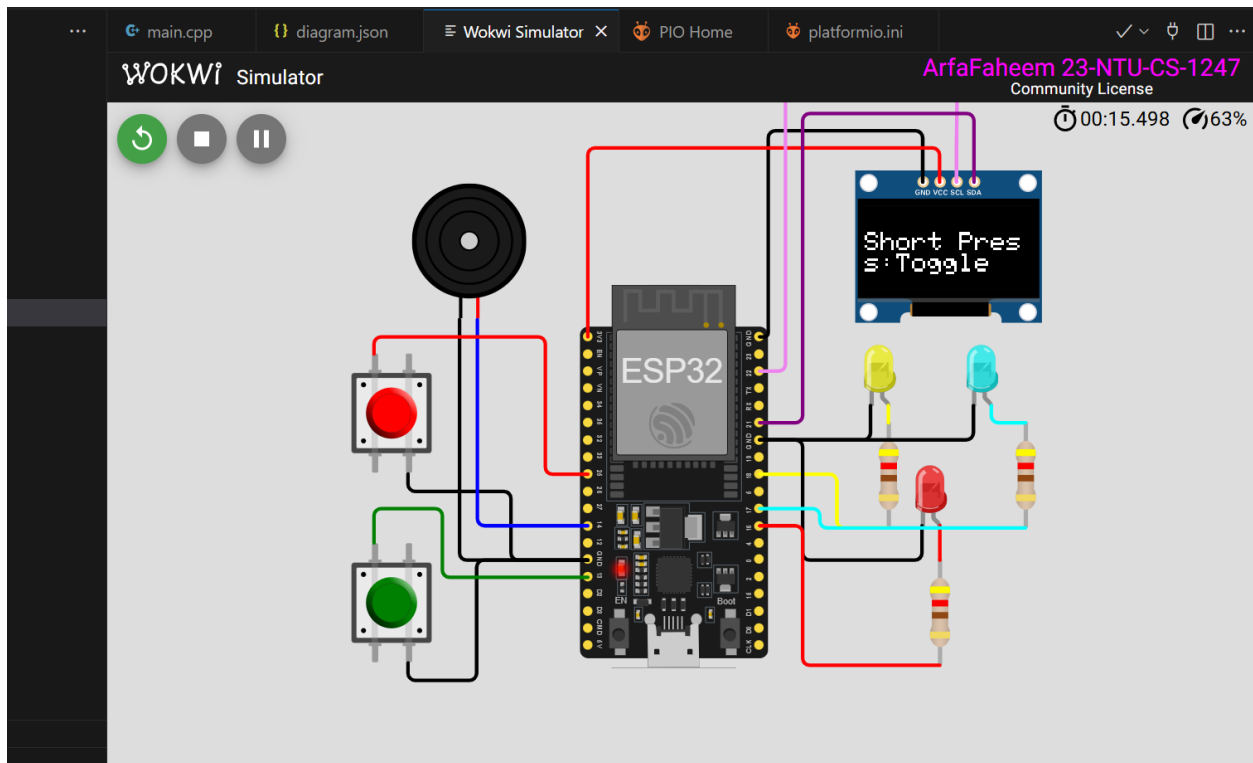
```
        delay(7);
```

```
}  
  
}  
  
}
```

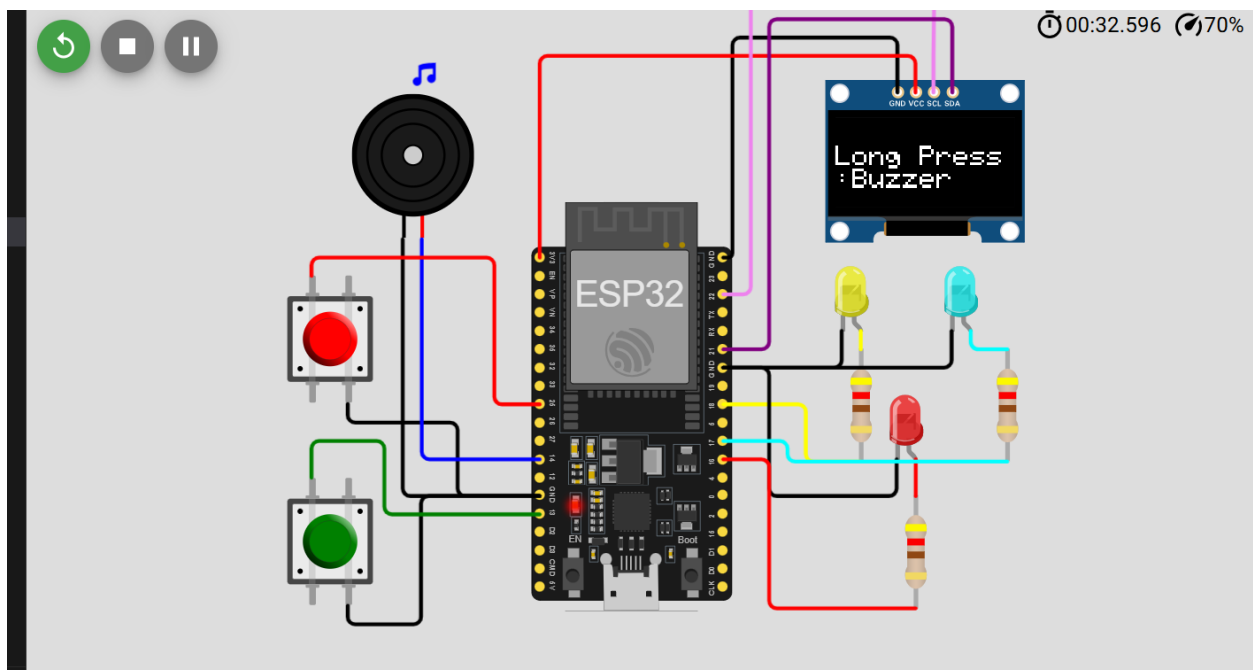
Task B — Coding: Use a single button with press-type detection (display the event on the OLED):

Short press – toggle





Long press- buzzer tone



Code for task B:

//Task B

```
//Name: Arfa Faheem RegNo: 23-NTU-CS-1247
```

```
//Embedded IOT Fall 2025
```

```
//BS AI 5TH
```

```
//Submitted to: Sir Nasir
```

```
#include <Arduino.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
// pins for led1 and buzzer , here i use only one led for toggling behaviour
```

```
#define BTN    25
```

```
#define LED1   16
```

```
#define BUZZER 14
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
#define OLED_ADDR 0x3C
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

```
bool ledState = LOW;
```

```
unsigned long lastButtonTime = 0;    // debounce timer
```

```
const unsigned long debounceDelay = 150; // ms
```

```
const unsigned long longPressTime = 1500; // ms
```

// this function is to show event on oled display whether its short one for toggle on long one for buzzer

```
void showEventOnOLED(const char* msg) {  
    display.clearDisplay();  
    display.setTextSize(2);  
    display.setTextColor(SSD1306_WHITE);  
    display.setCursor(0, 20);  
    display.println(msg);  
    display.display();  
}
```

```
void setup() {  
    Serial.begin(115200);  
    // button is set as input, led and buzzer as output  
    pinMode(BTN, INPUT_PULLUP);  
    pinMode(LED1, OUTPUT);  
    pinMode(BUZZER, OUTPUT);  
    digitalWrite(LED1, ledState);  
    digitalWrite(BUZZER, LOW); // initially buzzer is off
```

// PWM buzzer setup

```
ledcSetup(0, 2000, 10); // channel 0, 2 kHz frequency, 10-bit resolution
```

```
ledcAttachPin(BUZZER, 0);
```

// Initialize OLED

```
Wire.begin(21, 22);
```

```

if (!display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR)) {
    Serial.println("OLED initialization failed");
    while (1);
}

display.clearDisplay();
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(0,9);
display.println("Arfa's Task B");
display.display();
delay(1000);
}

void loop() {
    int buttonState = digitalRead(BTN); // it gets button state

    // Check if button pressed and debounce time passed (handled using millis())
    if (buttonState == LOW && (millis() - lastButtonTime > debounceDelay)) {
        lastButtonTime = millis();

        unsigned long pressStart = millis();

        // Wait until button released
        while (digitalRead(BTN) == LOW) { //loop continues till button is pressed
            delay(1);
        }
    }
}

```

```

unsigned long pressDuration = millis() - pressStart; //it calculates press duration

// this is for short press
if (pressDuration < longPressTime) {
    ledState = !ledState; //this is for toggling led, if led on then off on button press and vice versa
    digitalWrite(LED1, ledState); //updates led new state
    showEventOnOLED("Short Press:Toggle");
}

//this is for long press
else {
    showEventOnOLED("Long Press:Buzzer");
    for (int i = 0; i < 3; i++) { //it will play buzzer 3 times
        ledcWriteTone(0, 2000);
        delay(200);
        ledcWrite(0, 0);
        delay(100);
    }
}
}

```

Github Repo Link:

<https://github.com/arfafaheem/IOT-arfa47/tree/main>

Wokwi Project Links:

Task A: <https://wokwi.com/projects/445577561315698689>

Task B: <https://wokwi.com/projects/445803312390334465>